技术报告 · hands-on-policy-distillation

从第一性原理理解同策略蒸馏

两个小型 Qwen 模型学会使用工具

2026 年 10 月 · 一块 A100-80GB 上的 Qwen3-8B → Qwen3-1.7B 与 Qwen3.5-9B → Qwen3.5-2B

摘要

我们在一块 A100-80GB GPU 上，用同一套代码和超参数，把 Qwen3-8B 蒸馏进 Qwen3-1.7B，把 Qwen3.5-9B 蒸馏进 Qwen3.5-2B，共四个阶段：小学数学应用题、计算器工具、多轮回合中的三种工具，以及蒸馏加任务奖励。我们从“学生该模仿什么、在哪里模仿”出发，推导出同策略蒸馏就是一种策略梯度：每个 token 的奖励等于教师对学生所写 token 的对数概率减去学生自己的对数概率。每个蒸馏阶段的教师 KL 都在下降；在完全不用奖励的情况下，任务成功率也随之上升：计算器任务上从 0.47 升到 1.00、从 0.65 升到 0.90，三工具任务上从 0.66 升到 0.75、从 0.75 升到 0.96。再通过 GRPO 加入任务奖励，没有带来可测量的变化，因为分别有 98.5% 和 89% 的采样组得到完全相同的奖励。Qwen3.5 的训练用了 2.5 倍的 GPU 时间，原因是它的回合长 2–5 倍。

0.47 → 1.00	0.75 → 0.96	98.5%	2.5×
Qwen3 学生的计算器任务解决率，未用奖励	Qwen3.5 学生的三工具任务解决率，未用奖励	Qwen3 的 GRPO 组奖励完全相同：没有信号	Qwen3.5 的 GPU 时间，因其回合长 2–5 倍

1 引言

能调用工具的小模型部署成本低，但它得学会：什么时候调用哪个工具、调用怎么写、拿到结果之后怎么办。教会它有三条路：在更大的教师模型写出的回合上做监督微调（异策略蒸馏；Kim and Rush, 2016）；用任务奖励做强化学习，例如 GRPO (Shao et al., 2024)；以及同策略蒸馏（on-policy distillation）——学生自己写回合，教师给每个 token 打分 (Agarwal et al., 2024; Gu et al., 2024; Lu and Thinking Machines Lab, 2025)。Qwen3 的小模型本身就是用同策略蒸馏从更大的模型后训练出来的 (Yang et al., 2025a)。

本报告先从第一性原理解释第三条路为什么可行（第 2 节），再说明我们如何训练（第 3 节），以及两次训练的结果（第 4 节）。两次训练用的是同一条流水线（在 TRL 的 GRPO 训练器的优势中加入教师项）和同一套配置，只有模型不同。

2 从第一性原理看核心思想

设定 语言模型是定义在 token 上的策略。状态 s_t 是学生写第 t 个 token 之前上下文中的全部内容：提示（系统提示、工具说明、任务）、学生先前写下的 token，以及它调用过的工具返回的结果。动

作是下一个 token y_t。学生 π_θ 参与训练，教师 π_T 保持冻结。工具结果由环境写入，而不是学生选择的，所以它属于状态，永远不是动作。

我们希望学生在每个状态下都像教师那样选择 token。衡量两个下一 token 分布之间差异的自然选择是 KL 散度，于是只剩两个问题：在哪些状态上衡量，取哪个方向。

在哪些状态上：学生自己的状态 在教师写的回合上微调，训练的是教师会访问的状态。可到了测试时，生成的是学生，访问的是它自己的状态。只要写出一个教师不会写的 token，学生就进入了训练样本从未覆盖的状态，在那里更容易再次出错：误差会累积。在模仿学习中，若训练分布上每步的错误率为 ϵ，T 步的总代价可能按 $O(\epsilon T^2)$ 增长 (Ross and Bagnell, 2010)；改在学习者自己的策略产生的状态上训练、由专家标注，可将其降到 $O(\epsilon T)$ (DAgger; Ross et al., 2011)。同策略蒸馏就是以教师为专家的这套做法：学生写回合，教师在学生到达的每个状态上指出“换作我会怎么做”。

取哪个方向：反向 KL 目标函数是反向 KL，对学生自己的回合 τ 中的所有状态求和：

$$\mathscr{L}(\theta) = \mathbb{E}_x \, \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \mathrm{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_T(\cdot \mid s_t)\big)\Big]. \tag{1}$$

由 KL 的链式法则，这个求和的期望等于两个模型在完整回合上的分布之间的 KL；工具结果在两个模型下都是历史的同一个函数，相互抵消。反向 KL 适合把大模型蒸馏进小模型，理由有三。(i) 它是学生分布下的期望，可以用学生的样本和教师对这些 token 的对数概率来估计：教师只需一次前向传播，无须生成。(ii) 它是寻模（mode seeking）的：学生把概率放在教师几乎不放的地方会受重罚，漏掉教师的某些选项只受轻罚。无法表示大教师全部行为的学生，会专注于一种好的行为，而不是把几种行为平均起来 (Gu et al., 2024; Lu and Thinking Machines Lab, 2025)。(iii) 只有当学生在其访问的状态上表现得与教师一致时，它才达到最小值零，所以除了模仿，没有别的办法能降低它。与学习得到的奖励模型不同，它无法被钻空子；Lu and Thinking Machines Lab (2025) 称之为 unhackable。

梯度就是策略梯度 固定一个状态 s，记 $p = \pi_\theta(\cdot \mid s)$、$q = \pi_T(\cdot \mid s)$，$a$ 取遍词表。由于 $p(a)\nabla_\theta \log p(a) = \nabla_\theta p(a)$ 且 $\sum_a \nabla_\theta p(a) = \nabla_\theta 1 = 0$，

$$\begin{aligned}\nabla_\theta \mathrm{KL}(p \,\|\, q) &= \sum_a \nabla_\theta p(a)\big[\log p(a) - \log q(a)\big] + \sum_a \nabla_\theta p(a) \\ &= \mathbb{E}_{a\sim p}\Big[\big(\log p(a) - \log q(a)\big)\nabla_\theta \log p(a)\Big].\end{aligned} \tag{2}$$

对学生实际采样到的 token y_t，定义

$$k_t = \log \pi_\theta(y_t \mid s_t) - \log \pi_T(y_t \mid s_t). \tag{3}$$

由式 (2)，$k_t \nabla_\theta \log \pi_\theta(y_t \mid s_t)$ 是 s_t 处 KL 梯度的单样本无偏估计。整个方法就是这一句：

算法 1　一个训练步

输入：　学生 π_θ（LoRA）、冻结的教师 π_T、环境、提示 $x_1, \ldots, x_P$

for 每个提示 x_p **do**
　用 π_θ（vLLM）采样 G 个回合，执行每个工具调用并追加其结果
　$R_{p,i} \leftarrow$ 回合 i 的任务奖励；　$\hat{A}_{p,i} \leftarrow (R_{p,i} - \bar{R}_p)/\sigma_p$　▷ $G = 1$ 时为 0
end for
教师对每个回合做一次前向传播：$k_t \leftarrow \log \pi_\theta(y_t \mid s_t) - \log \pi_T(y_t \mid s_t)$
对回合 (p, i) 的每个学生 token t：$A_t \leftarrow c_R \hat{A}_{p,i} - c_T k_t$
对 $\ell(\theta) = -\frac{1}{N}\sum_t A_t \log \pi_\theta(y_t \mid s_t)$ 做一步 AdamW　▷ N 个学生 token

各阶段只在环境、组大小 G 和 (c_T, c_R) 上不同。

核心等式

在学生自己的样本上最小化反向 KL (1)，就是 REINFORCE (Williams, 1992)：最小化替代损失

$$\ell(\theta) = -\frac{1}{N}\sum_t A_t \log \pi_\theta(y_t \mid s_t), \qquad A_t = -k_t \text{（视为常数）}, \tag{4}$$

求和遍历一个批次中的 N 个学生 token。token t 的奖励为

$$r_t = \log \pi_T(y_t \mid s_t) - \log \pi_\theta(y_t \mid s_t). \tag{5}$$

这个奖励等于教师对学生所写内容的对数概率，加上学生自己的自信息（surprisal，相当于熵奖励）。教师比学生更认可的 token 被推高，不如学生认可的被压低。对 token 取平均后，k_t 也是每 token 反向 KL 的无偏估计，即图 1 中的教师 KL 曲线。

两处近似　第一，$\mathscr{L}$ 还通过状态依赖于 θ：每个 token 都会影响其后的状态，式 (1) 的精确梯度要在 A_t 上再加上未来项 $\sum_{t'>t} k_{t'}$。我们与 Lu and Thinking Machines Lab (2025) 一样去掉这些项（折扣因子取零）：每个 token 只为自己的分歧负责，这降低了方差，而下一批数据会在更新后的学生所访问的状态上继续训练。第二，用一个采样 token 代替式 (2) 中对整个词表的求和。GKD (Agarwal et al., 2024) 精确计算这个和，需要两个模型在每个位置上的完整下一 token 分布（Qwen3.5 每个位置 248,320 个数）；采样估计只需要教师给出一个数。

为什么稠密信号胜过稀疏信号　对比用二值任务奖励 $R \in \{0, 1\}$ 的强化学习。GRPO 为每个提示采样一组 G 个回合，并给回合 i 的每个 token 相同的优势 $\hat{A}_i = (R_i - \bar{R})/\sigma_R$（在组内归一化）。由此得出两点。一个回合的奖励至多携带一比特信息，还要分摊到几百个 token 上：它只说明回合失败了，不说明错在哪里。而且，若一个提示的 G 个回合奖励全部相同，组内所有优势都为零。若学生以概率 p 解出某个提示、且各样本相互独立，则一组带有信号的概率为

$$P_{\text{signal}}(p) = 1 - p^G - (1-p)^G, \tag{6}$$

当 $G = 4$ 时，它在 $p = \frac{1}{2}$ 处为 0.875，在 $p = 0.955$ 处为 0.17，p 趋于 0 或 1 时趋于零。教师项则为每个 token 提供一个数，每个回合 $O(T)$ 比特而非 $O(1)$ (Lu and Thinking Machines Lab, 2025)；只要学

表 1 两组教师–学生组合，其余一切相同。

	Qwen3 训练	Qwen3.5 训练
教师（冻结，bf16）	Qwen3-8B	Qwen3.5-9B（仅语言模型）
学生	Qwen3-1.7B	Qwen3.5-2B
学生层数	28 层，全为注意力	18 层 Gated DeltaNet + 6 层门控注意力
词表大小	151,936	248,320
工具调用格式	JSON	XML
可训练 LoRA 参数	34.9 M	33.6 M

生与教师有分歧它就不为零，无论回合成败。它也很便宜：教师用一次前向传播（各位置并行）为整个回合打分，从不解码。

结合两种信号 一般地，回合 i 中 token t 的优势为

$$A_t = c_R \hat{A}_i - c_T k_t, \tag{7}$$

纯蒸馏时 $c_R = 0$。教师项至多让学生与教师一样好；只有任务奖励能让学生超越教师或纠正教师的错误，而且只能通过式 (6) 留下的那些有信号的组。算法 1 是每个阶段的一个训练步。

前提条件 k_t 比较的是两个模型对同一个 token id 的概率，所以学生和教师必须使用相同的分词器：Qwen3（151,936 个 token）与 Qwen3.5（248,320 个）不能混用，这就是每个系列只在自身内部蒸馏的原因。提示和工具结果进入 s_t，但不计入损失。此外，教师必须评判它自己永远不会到达的状态，例如学生犯错之后的状态。

3 训练设置

模型与硬件 表 1 列出两组教师–学生组合。Qwen3.5 (Qwen Team, 2026) 以三比一的比例交替使用 Gated DeltaNet 层 (Yang et al., 2025b)（一种线性注意力）和门控注意力层。两次训练都使用非思考模式；在学生语言模型的每个线性层上加秩为 32、$\alpha = 32$ 的 LoRA (Hu et al., 2022)（Qwen3.5 的检查点还带有视觉塔，这些纯文本任务用不到它）；都只用一块 Colab A100-SXM4-80GB。训练器是 TRL 的 `GRPOTrainer` (von Werra et al., 2020)，用 vLLM 生成 (Kwon et al., 2023)。我们的子类在每批采样之后计算 k_t，并用式 (7) 替换 GRPO 的优势；采样、工具执行、掩码和损失都沿用 TRL。

课程 表 2 列出四个阶段。阶段 1 采样 GSM8K 题目 (Cobbe et al., 2021)，参考答案从不进入损失。阶段 2 的应用题数字足够大（比如五位数乘四位数），用文字心算常常出错。阶段 3 和 4 在每个任务上都提供三种工具：计算器、在随机生成的公司数据库上执行只读 SQL，以及在随机生成的商店文档上执行 JMESPath 查询。这些任务需要一种工具（45%）、组合两种工具（40%），或不需要工具（15%，应直接作答）。阶段 2–4 的任务由程序生成，训练与评估使用不同的随机种子。每个阶段都在上一阶段的 LoRA 适配器上继续训练。

表 2 课程设置，两次训练完全相同。Token 上限：提示之后整个回合的长度上限，含工具结果。批次：每步的回合数，即提示数 × 组大小 G。

阶段	任务	工具	轮数	Token 上限	步数	批次	学习率	(c_T, c_R)
1	GSM8K	无	1	1,024	200	32×1	10^{-4}	(1, 0)
2	大数算术	计算器	4	2,048	150	32×1	10^{-4}	(1, 0)
3	一种、两种或不用工具	全部三种	6	3,072	200	32×1	10^{-4}	(1, 0)
4	同阶段 3	全部三种	6	3,072	150	16×4	$5 \cdot 10^{-5}$	(1, 1)

优化与打分

- 每个优化步都使用由当前权重新采样的一批数据，因此 PPO 比率为 1，裁剪（下限 $\epsilon = 0.2$，上限 0.28）从不起作用，更新就是式 (4) 的梯度，在批次内所有学生 token 上取平均（TRL 的 DAPO 损失；Yu et al., 2025）。
- AdamW，5 步预热后学习率保持不变，梯度范数裁剪到 1，无权重衰减；每个微批 8 个回合，使用梯度检查点。10^{-4} 约为全参数微调学习率的十倍，这是 LoRA 的推荐做法 (Schulman and Thinking Machines Lab, 2025)。
- 采样：vLLM 与训练共用 GPU（占 30% 显存），温度 1，top-p 为 1。
- 打分：教师对每个回合做一次 bf16 前向传播，经由融合的输出层直接返回采样 token 的对数概率，不生成完整的 logits。k_t 中学生的对数概率由训练器重新计算，而不取自 vLLM。
- vLLM 的算子给出的对数概率与训练器略有差异，因此 TRL 给每个回合乘以重要性权重 $\exp \sum_t (\log \pi_\theta - \log \pi_{\text{vLLM}})$，并丢弃权重超过 3 的回合；各阶段的平均权重为 0.95–1.00。
- 奖励（阶段 4 的训练与全部评估）：若最终回答中最后一个 \boxed{} 与参考答案相等（数值误差 10^{-6} 以内）则为 1，否则为 0；若回合用完 token 上限，或到达轮数上限时仍在调用工具，也记为 0。
- 每 25 步在 256 个留出任务上评估（阶段 1 用 GSM8K 测试集题目），每个任务以温度 1 采样一次，因为 TRL 没有贪心解码的评估。

4 结果

表 3 给出每个阶段第一次和最后一次评估的结果；图 1 给出全部评估。

蒸馏的效果与推导一致 每个蒸馏阶段的教师 KL 都下降了 23%–83%，且大部分降幅出现在前 25–50 步（Qwen3 阶段 2：到第 25 步已从 0.328 降至 0.049）。

不用奖励，任务成功率也随之上升 阶段 1–3 从未见过奖励，但每个阶段的评估奖励都在上升：GSM8K 上分别提高 0.03 和 0.13，计算器任务上提高 0.53 和 0.26，三工具任务上提高 0.09 和 0.20（依次为 Qwen3 和 Qwen3.5）。学生学会了教师使用工具的习惯。在阶段 2 的第一批数据中，Qwen3.5 学生在 32 个回合里只调用了 4 次计算器，其余算术都用文字完成；在最后 50 步中，它平均每个回

表 3 每个阶段开始和结束时的评估（256 个留出任务，温度 1），以及含评估在内的实际耗时。

	Qwen3-8B → Qwen3-1.7B			Qwen3.5-9B → Qwen3.5-2B		
阶段	教师 KL	奖励	耗时	教师 KL	奖励	耗时
1. GSM8K	0.229 → 0.136	0.809 → 0.836	45 分钟	0.218 → 0.168	0.656 → 0.781	85 分钟
2. 计算器	0.328 → 0.057	0.473 → 1.000	18 分钟	0.108 → 0.047	0.645 → 0.902	103 分钟
3. 三种工具	0.378 → 0.068	0.660 → 0.746	52 分钟	0.163 → 0.034	0.754 → 0.957	109 分钟
4. + 任务奖励	0.065 → 0.067	0.754 → 0.758	55 分钟	0.034 → 0.025	0.945 → 0.953	130 分钟
合计			2.8 小时			7.1 小时

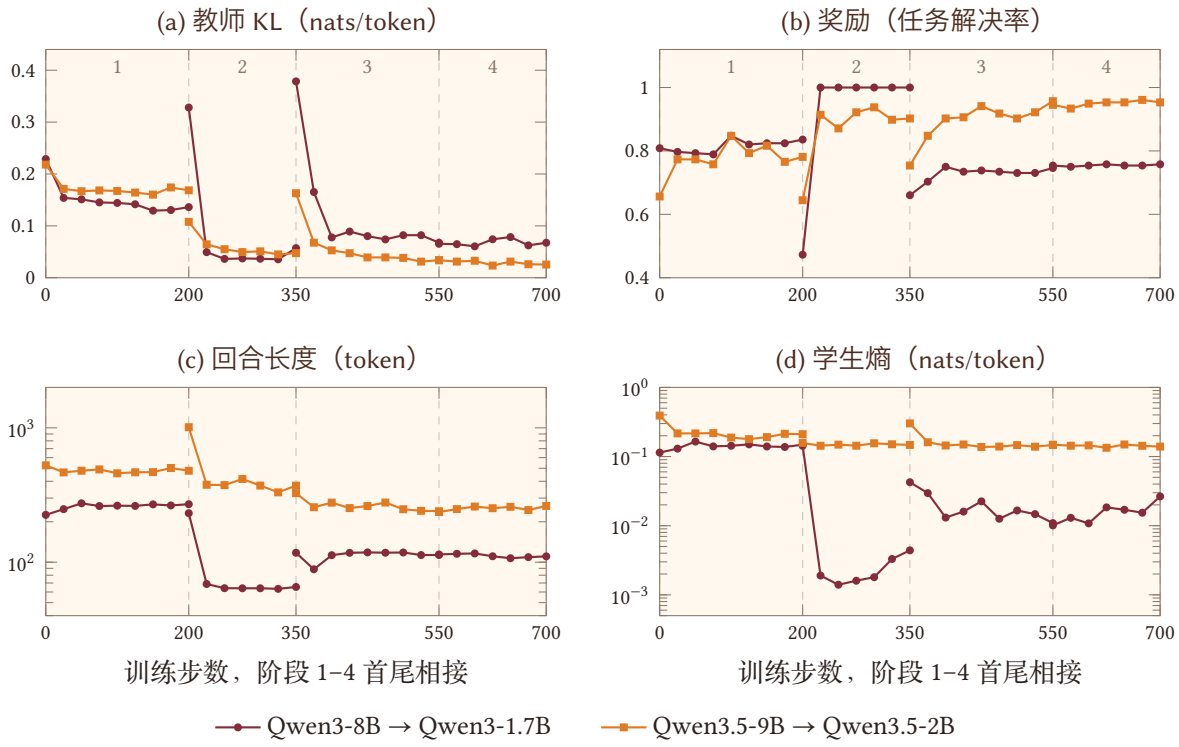


图 1 四个阶段首尾相接训练期间的评估。虚线分隔各阶段；每个阶段开头的评估用的是上一阶段的权重，但在新任务上进行。(a) 学生 token 上 k_t 的均值。(b) 256 个留出任务的解决比例。(c) 提示之后回合的平均长度（对数坐标）。(d) 学生下一 token 分布的平均熵（对数坐标）。

合调用 1.55 次（Qwen3：第一批 32 个回合调用 15 次，最后每回合 1.16 次）。在阶段 3，实际执行的工具调用中返回错误的比例从 21% 降到 4%（前五步对比最后五步；Qwen3：从 14% 降到 7%）。

拖累 Qwen3.5 的是长度，而不是准确率 在 GSM8K 上，Qwen3.5 学生有 10%–18% 的回答超过 1,024 token 的上限而得零分。这些并非死循环：遇到一个别扭的中间结果（比如算出的人数带小数）后，模型开始反复自我怀疑。在写完的回答中，阶段 1 结束时有 91% 是对的（Qwen3：84%）。在阶段 2 的最后 50 步中，它约 70% 的失败训练回合用完了全部四轮，都在用更多的计算器调用反复核对结果。

任务奖励没有带来可测量的提升 阶段 4 的评估奖励从 0.754 变为 0.758，从 0.945 变为 0.953。成功率约 0.95 时，256 个任务的估计标准误为 0.014，而同一组 Qwen3.5 权重在阶段 3 结束时得分 0.957。式 (6) 解释了原因：只有 1.5%（Qwen3）和 10.9%（Qwen3.5）的组存在奖励方差。若一个提示的四个结果是按平均成功率（0.80 和 0.955）独立抽取的，式 (6) 预测的比例应为 59% 和 17%。Qwen3 学生的结果远比这确定：它一旦做错某个提示，四个样本就全错。它的熵在阶段 2 崩塌（从 0.14 降到 0.002 nats/token），此后一直低于 0.05；Qwen3.5 学生从阶段 2 起保持在 0.13–0.16。这正是反向 KL 所预言的：在其最小值处，学生的下一 token 分布（连同熵）等于教师的，所以自信的教师教出自信的学生（我们没有测量教师的熵），而自信的学生让 GRPO 无从比较。Qwen3 训练的阶段 5（只用 GRPO，$G = 8$，没有教师）证实了这一点：第一批之后每个组的方差都为零，损失和梯度恰好为零，我们在大约第 20 步停止了训练。

成本 Qwen3.5 的训练用了 7.1 小时，Qwen3 用了 2.8 小时（约 50 对 23 个 Colab 计算单元）。两次训练中每个 token 的成本相同：在阶段 4，按平均回合长度折算，每步每 token 分别耗时 0.17 秒和 0.16 秒。但 Qwen3.5 学生写的回合长 2–5 倍，每次调用工具前后都要用一句话解释（阶段 2：371 对 65 个 token；阶段 3：240 对 113）。

5 要点

1 蒸馏是主力

只用提示，没有标注也没有奖励，就能教会 1.7B 或 2B 的学生使用工具：每个阶段在一块 A100 上不到两小时，大部分提升出现在前 50 步。

2 强化学习需要能力边缘的任务

GRPO 只能从各样本结果不一的提示中学习。蒸馏消除了随机性，而我们的任务大多已被解决；更难的任务、更大的组或更高的温度，才是可用的杠杆。

3 预算要按模型定

配置相同，但 Qwen3.5 写的回合长 2–5 倍：对 Qwen3 从未起作用的上限让它损失了准确率，训练也用了 2.5 倍的 GPU 时间。

4 分词器决定谁能当教师

教师必须与学生逐个 token 共享同一个词表，所以每个系列只能在自身内部蒸馏。

复现 阶段 1–4 用以下命令运行：

```
BACKEND=trl EXTRA=--use_vllm scripts/run_stages.sh a100 1 4
```

训练器是 `opd/trl_train.py`，配置在 `opd/configs/`。所有记录的指标、采样回合和适配器都在 `results/trl/a100/`（Qwen3）和 `results/trl/a100-qwen3.5/`（Qwen3.5）中。

参考文献

R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

Y. Gu, L. Dong, F. Wei, and M. Huang. MiniLLM: Knowledge distillation of large language models. In *ICLR*, 2024.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Y. Kim and A. M. Rush. Sequence-level knowledge distillation. In *EMNLP*, 2016.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

K. Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi:10.64434/tml.20251026.

Qwen Team. Qwen3.5 model cards. Hugging Face, 2026.

S. Ross and J. A. Bagnell. Efficient reductions for imitation learning. In *AISTATS*, 2010.

S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.

J. Schulman and Thinking Machines Lab. LoRA without regret. *Thinking Machines Lab: Connectionism*, 2025.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, S. Huang, K. Rasul, and Q. Gallouédec. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025a.

S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *ICLR*, 2025b.

Q. Yu et al. DAPO: An open-source LLM reinforcement learning system at scale. arXiv:2503.14476, 2025.